Значимость различий в качестве классификаторов

Наивные и корректные методы проверки гипотез при сравнении моделей на нескольких наборах данных

Кирилл Дмитриев

Курсовой проект по дисциплине «Статистический анализ данных»

Разница в точности — случайная величина

Обычно смотрят, у какой модели число больше, и считают это доказательством.

Но точность зависит от того, на каких объектах модель обучали и проверяли. Чтобы утверждать «A лучше B», нужна проверка гипотезы.

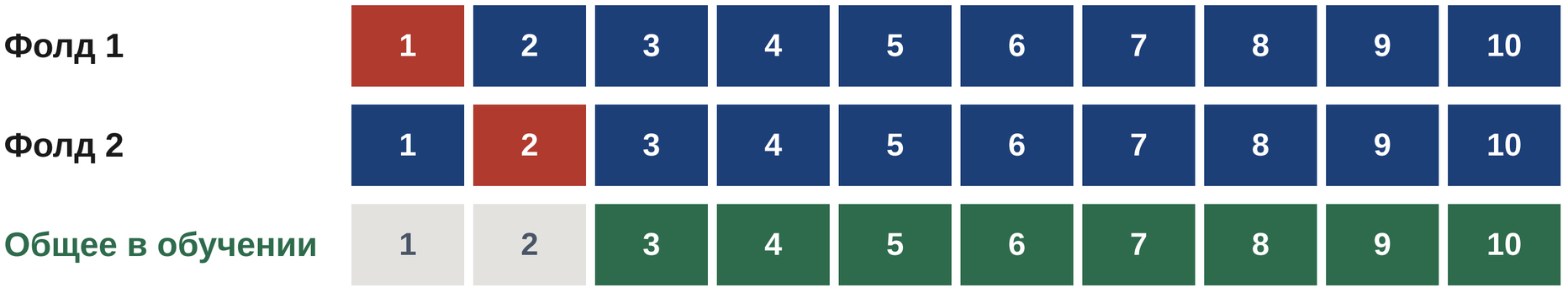
≈3%

из 121 работы NeurIPS 2017 в обзоре сообщали о формальном сравнении алгоритма с baseline

Király, Mateen, Sonabend (2018), arXiv:1812.07519

Почему очевидный тест сомнителен

Данные делятся на 10 частей. В каждом фолде одна часть — **проверка**, остальные девять — **обучение**.



Обучение Фолда 1 и Фолда 2 совпадает на **8 частях из 9**. Результаты фолдов связаны, а t-test считает их независимыми.

Цель и план работы

Показать на расчётах, когда наивный и корректные тесты расходятся и насколько.

H_0: у двух моделей одинаковое качество. Маленькое p (меньше 0.05) говорит, что случайность это плохо объясняет.

1. Четыре теста на одной паре, затем на 45 сравнениях
2. Групповое сравнение: критерий Фридмана и пост-тест Неменьи
3. Поправка Холма на множественные сравнения
4. Доверительные интервалы для размера эффекта
5. Симуляция: ошибка первого рода и мощность

Что такое классификатор и как читать таблицу

Классификатор — программа, которая по описанию объекта угадывает, к какой группе он относится. Например, письмо: спам или не спам.

Объект

строка таблицы

Одно письмо, один пациент, одна банкнота. Всё, что мы хотим отнести к группе.

Признак

столбец таблицы

Измеренная характеристика объекта: возраст, давление, сколько раз встречается слово «скидка».

Класс

правильный ответ

Группа, которую нужно угадать: «спам» / «не спам», «болен» / «здоров», цифра от 0 до 9.

Модель сначала учится на объектах, ответ для которых известен, потом отвечает для новых. Точность (accuracy) — доля верных ответов: 0.95 значит 95 верных из 100.

Датасеты: что в них предсказываем (1/2)

Датасет	Что определяем и по чему	Объектов	Признаков	Классов
digits	Рукописная цифра от 0 до 9 по картинке 8×8 пикселей	1797	64	10
breast_cancer	Опухоль злокачественная или нет, по замерам клеток	569	30	2
wine	Сорт итальянского вина (из трёх) по химическому анализу	178	13	3
banknote	Настоящая или поддельная банкнота, по её фотографии	1372	4	2
credit_g	Надёжный ли заёмщик банка, по анкете клиента	1000	20	2
diabetes	Есть ли диабет, по медицинским анализам	768	8	2
vehicle	Тип автомобиля (4 вида) по форме силуэта на снимке	846	18	4
segment	Что на фрагменте фото: небо, трава, окно, кирпич и т. д.	2310	19	7

Датасеты: что в них предсказываем (2/2)

Датасет	Что определяем и по чему	Объектов	Признаков	Классов
ionosphere	Радарный сигнал: есть ли в ионосфере структура	351	34	2
glass	Тип стекла (6 видов) по химическому составу	214	9	6
heart_statlog	Есть ли болезнь сердца, по обследованию пациента	270	13	2
spambase	Спам ли письмо, по частоте слов и символов	4601	57	2
phoneme	Носовой или ротовой гласный звук, по записи речи	5404	5	2
blood	Придёт ли донор сдавать кровь снова, по истории визитов	748	4	2
sonar	Мина или камень, по эху гидролокатора	208	60	2

Источники: sklearn и OpenML. В credit_g 13 признаков текстовые (например, цель кредита): они переводятся в 0/1-столбцы (one-hot) внутри модели, в каждом фолде отдельно.

Код: загрузка данных

```
datasets = {}
for name, loader in [("digits", load_digits),
                     ("breast_cancer", load_breast_cancer),
                     ("wine", load_wine)]:
    X, y = loader(return_X_y=True, as_frame=True)
    datasets[name] = (X, y.to_numpy())

for short_name, openml_name in openml_names.items():
    X, y = fetch_openml(openml_name, version=1,
                        return_X_y=True, as_frame=True)
    datasets[short_name] = (X, y.to_numpy())
```

Каждый датасет это пара (X, y): таблица признаков и метки классов.

openml_names: словарь вида {"credit_g": "credit-g", ...} с названиями на сайте OpenML.

Размеры первых датасетов:

	объектов	признаков
digits	1797	64
wine	178	13
credit_g	1000	20

Три модели простыми словами

LogisticRegression

линейная модель

Каждому признаку присваивается «вес важности», взвешенная сумма превращается в вероятность класса. Несмотря на слово regression в названии, это классификатор.

RandomForest

случайный лес

Сотня деревьев решений: каждое — цепочка вопросов «да/нет», например «давление выше 130?». Деревья обучены на разных частях данных, ответ — голосование.

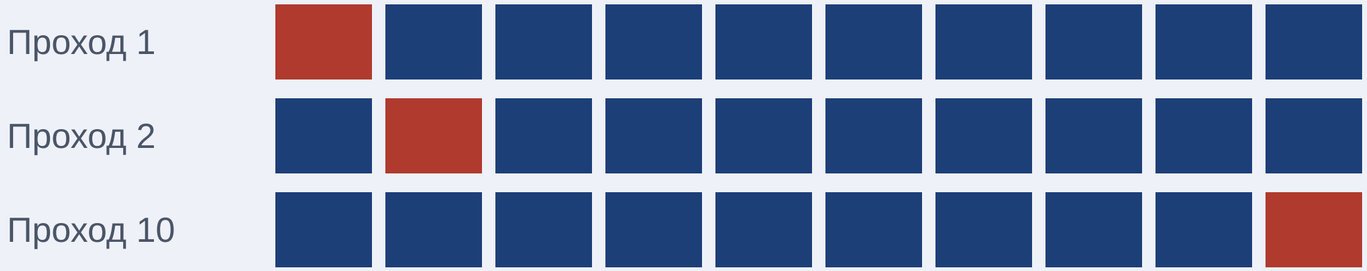
HistGradientBoosting

градиентный бустинг

Деревья строятся по очереди: каждое новое исправляет ошибки предыдущих. Hist — значения признаков группируются в корзины, чтобы считать быстро.

Три разных подхода: простая формула против двух сильных ансамблей деревьев. Ансамбль — когда ответ складывается из многих простых моделей.

Как измеряем качество: 10-fold кросс-валидация



10 частей данных: красная — проверка, синие — обучение. Так 10 раз.

- Метрика — accuracy: доля верных ответов на объектах, которых модель не видела при обучении.
- Настройки моделей стандартные, без подгонки; random_state = 42 фиксирует «случайность», чтобы результат повторялся.
- Масштабирование и кодирование признаков делаются внутри каждого прохода, чтобы проверочные данные не «подсматривали».

Код: модели и кросс-валидация

```
def make_models(X):
    return {
        "logreg": make_pipeline(
            make_preprocessor(X),
            LogisticRegression(max_iter=1000, random_state=42)),
        "rf": make_pipeline(
            make_preprocessor(X),
            RandomForestClassifier(random_state=42)),
        "hgb": make_pipeline(
            make_preprocessor(X),
            HistGradientBoostingClassifier(random_state=42)),
    }
def cv_scores(ds_name, model_name):
    X, y = datasets[ds_name]
    model = make_models(X)[model_name]
    return cross_val_score(model, X, y, cv=10)
```

make_preprocessor: пропуски заполняются медианой, числа стандартизуются, категории кодируются в 0/1. Всё это внутри конвейера, поэтому учится заново в каждом фолде.

cv_scores для RF на digits: 10 значений accuracy

```
[0.900 0.972 0.950 0.939 0.956
0.978 0.972 0.972 0.933 0.927]
```

Цикл по всем парам «датасет × модель» выполняется параллельно по ядрам процессора.

Базовая точность моделей

	logreg	rf	hgb
digits	0.933	0.950	0.944
breast_cancer	0.981	0.963	0.968
wine	0.983	0.983	0.967
banknote	0.981	0.993	0.995
credit_g	0.750	0.762	0.754
diabetes	0.772	0.756	0.737
vehicle	0.799	0.749	0.781
segment	0.932	0.978	0.983
ionosphere	0.869	0.931	0.923
glass	0.654	0.786	0.753
heart_statlog	0.841	0.822	0.800
spambase	0.917	0.940	0.944
phoneme	0.751	0.914	0.895
blood	0.769	0.663	0.681
sonar	0.669	0.717	0.755

Лидер меняется от датасета к датасету: LogReg лучшая на 5, RF на 5, HGB на 4 (на wine ничья).

Разрывы разные: от долей процента до 16 п. п. (phoneme).

Различить случайный разрыв и настоящий как раз и помогают тесты.

Четыре способа проверить одну гипотезу

Тест	Как устроен	Особенность
Наивный t-test	t на 10 разностях точности по фолдам	Считает фолды независимыми
Corrected t-test (Nadeau и Bengio)	Дисперсия умножается на 1/k + n_test/n_train	Учитывает пересечение обучающих выборок
5×2cv (Dietterich)	5 раз делим пополам: одна половина обучение, другая проверка	Обучающая и тестовая части не пересекаются
McNemar	Одно разбиение, берём только объекты, где модели расходятся (точный тест, если их меньше 25)	Проверяет симметрию ошибок, а не среднюю точность

RF против LogReg на digits: четыре p-value

0.026
наивный t-test

0.100
corrected t-test

0.140
5×2cv t-test

0.118
McNemar

Только наивный тест отвергает H_0 (p меньше 0.05). Три остальных значимой разницы не находят. McNemar: модели расходятся лишь на 15 объектах из 450, 4 против 11.

Код: наивный, corrected и 5×2cv

```
a = results["digits"]["rf"]
b = results["digits"]["logreg"]

t_naive, p_naive = stats.ttest_rel(a, b)

p_left, p_right = CorrelatedTTest.probs(a, b)
p_corr = 2 * min(p_left, p_right)

_, p_5x2 = paired_ttest_5x2cv(
    rf, logreg, X, y, random_seed=42)
```

a и b: 10 значений accuracy по фолдам для RF и LogReg на digits. CorrelatedTTest из библиотеки baycomp сам применяет поправку Nadeau и Bengio.

Результат:

```
p_naive = 0.0259
p_corr = 0.0999
p_5x2 = 0.1400
```

Поправка увеличила p почти в 4 раза, и вывод поменялся.

Код: тест Макнемара

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
fit_a = clone(model_a).fit(X_tr, y_tr)
fit_b = clone(model_b).fit(X_tr, y_tr)

tb = mcnemar_table(
    y_target=y_te,
    y_model1=fit_a.predict(X_te),
    y_model2=fit_b.predict(X_te))
n_disagree = tb[0, 1] + tb[1, 0]
_, p = mcnemar(ary=tb, corrected=True,
    exact=bool(n_disagree < 25))
```

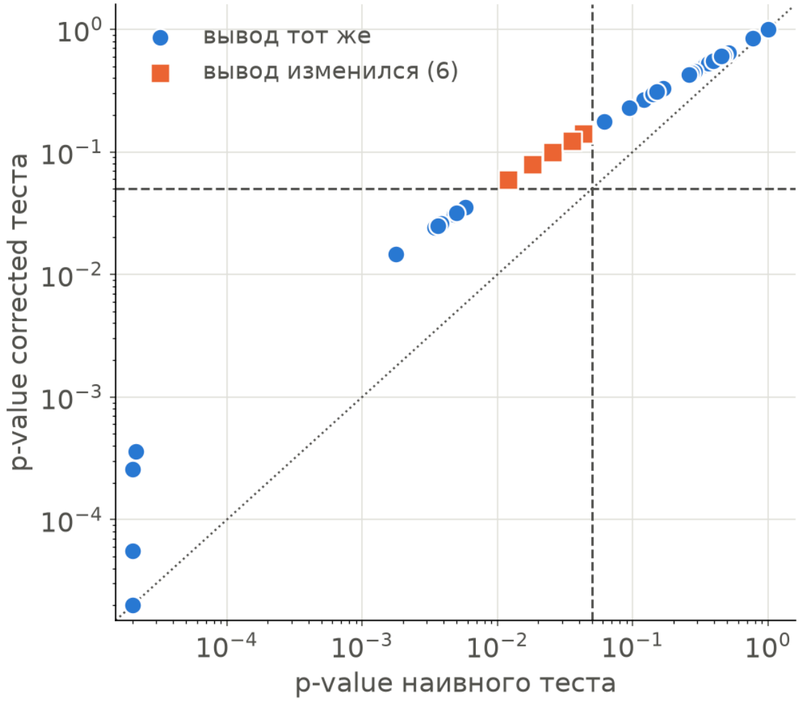
Диагональ таблицы: модели согласны (429 и 6). Вне диагонали: расходятся (4 и 11).

Результат:

```
[[429   4]
 [ 11   6]]
p = 0.118
```

Расхождений меньше 25, поэтому использован точный тест.

45 сравнений: наивный против corrected



Тест	Значимо из 45
Наивный	20
Corrected	14
5×2cv	9
McNemar	7

Corrected p больше наивного во всех 45 случаях. В 6 случаях (оранжевые) вывод меняется со «значимо» на «не значимо».

Код: функция сравнения пар

```
def compare_pair(ds_name, name_a, name_b):
    X, y = datasets[ds_name]
    models = make_models(X)
    sa = results[ds_name][name_a]
    sb = results[ds_name][name_b]

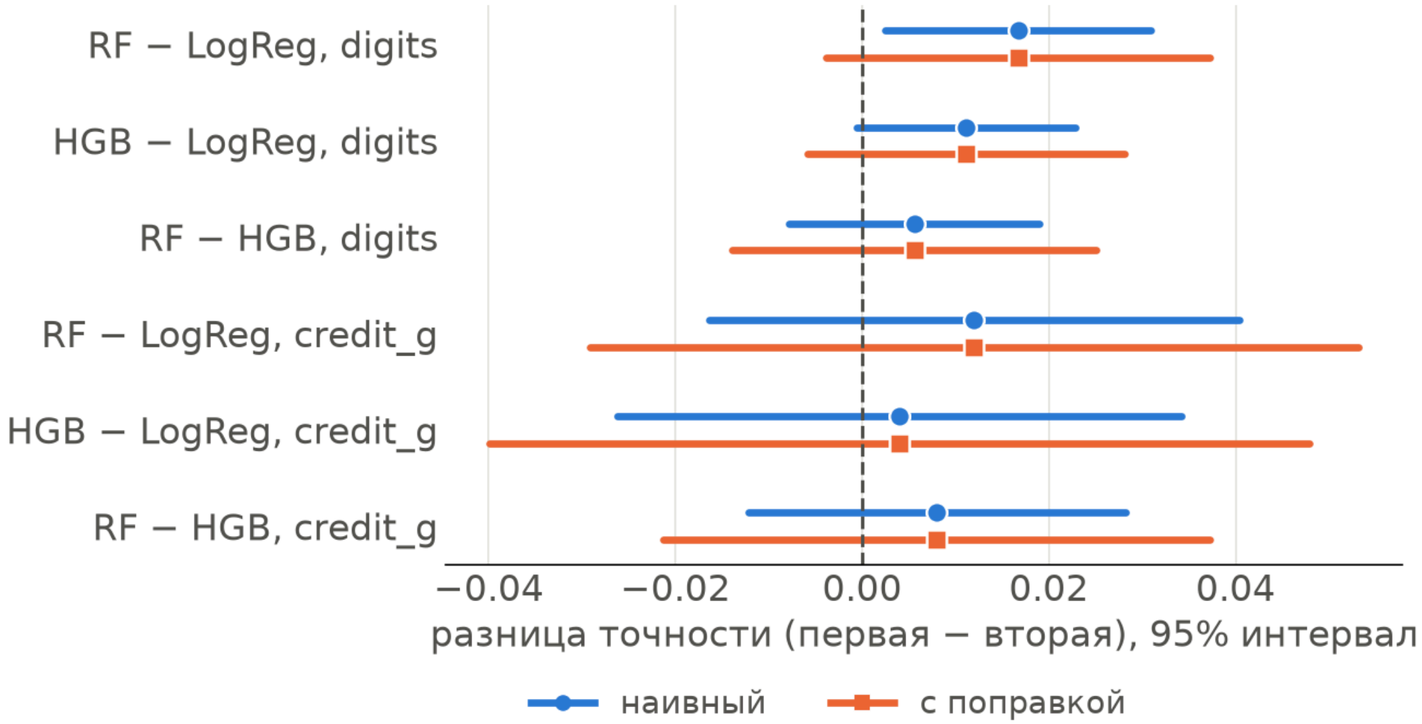
    p_naive = stats.ttest_rel(sa, sb).pvalue
    p_corr = corrected_ttest_p(sa, sb)

    p_5x2 = np.median([
        five_by_two_p(models[name_a],
        models[name_b], X, y, s)
        for s in range(5)])
```

```
    p_mc = np.median([
        mcnemar_p(models[name_a],
        models[name_b], X, y, s)[0]
        for s in range(10)])

    return {"dataset": ds_name,
            "pair": f"{name_a} vs {name_b}",
            "p_naive": p_naive,
            "p_corrected": p_corr,
            "p_5x2cv": p_5x2,
            "p_mcnemar": p_mc}
```

Размер эффекта и неопределённость



Наивный интервал не накрывает 0 только у одной пары из шести.

С поправкой не накрывает ни один.

Все разницы малы: 0.4–1.7 п. п.

Код: доверительные интервалы

```
diff = results[ds][a] - results[ds][b]
n = len(diff)
mean = diff.mean()
tcrit = stats.t.ppf(0.975, n - 1)
se_naive = np.sqrt(np.var(diff, ddof=1) / n)

_, var_corr, _ = CorrelatedTTest.compute_statistics(
    results[ds][b], results[ds][a])
se_corr = np.sqrt(var_corr)

lo_n, hi_n = mean - tcrit * se_naive, mean + tcrit * se_naive
lo_c, hi_c = mean - tcrit * se_corr, mean + tcrit * se_corr
```

diff: разности точности по 10 фолдам. tcrit: множитель для 95% интервала (t-распределение, 9 степеней свободы). Дисперсию с поправкой возвращает baycomp.

RF минус LogReg на digits:

	нижняя	верхняя
наивный	0.0025	0.0309
с поправкой	-0.0039	0.0373

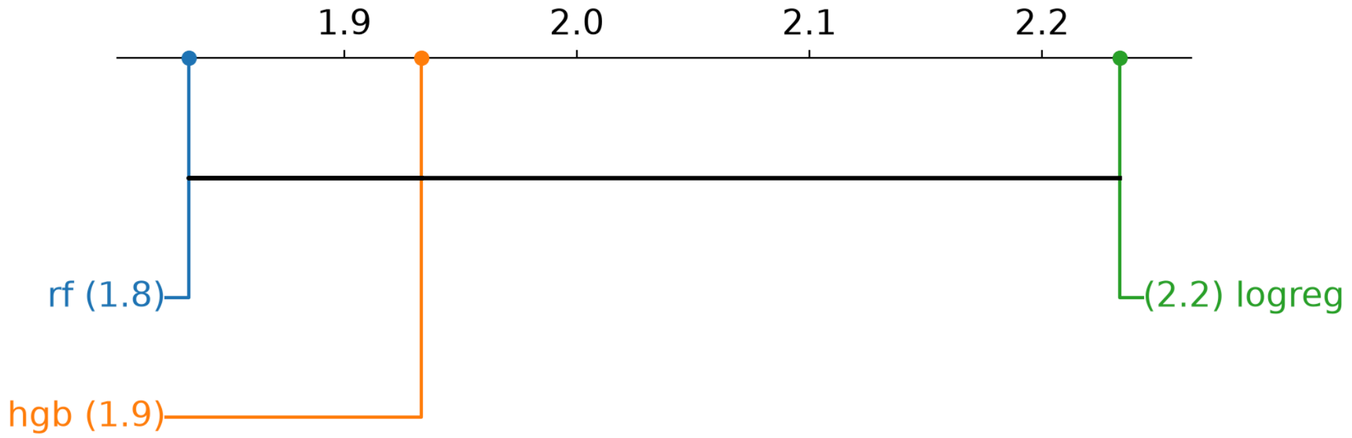
У наивного интервала ноль снаружи, у corrected внутри.

Три модели на пятнадцати датасетах

Фридман: $\chi^2 = 1.32$, $p = 0.52$.
Различий между моделями нет.

Ранги: rf 1.83, hgb 1.93, logreg 2.23.

Неменьи: все попарные p от 0.52 до 0.96.



Все три модели соединены одной перемычкой: статистически неразличимы. Единица наблюдения здесь датасет, а не фолд.

Код: Фридман, Неменьи, ранги

```
stat_f, p_f = friedmanchisquare(
    *[summary[m] for m in summary.columns])

nemenyi = sp.posthoc_nemenyi_friedman(summary.values)

ranks = summary.apply(
    lambda row: rankdata(-row), axis=1, result_type="expand")
avg_ranks = ranks.mean(axis=0)

sp.critical_difference_diagram(avg_ranks, nemenyi)
```

rankdata(-row): ранг 1 получает модель с наибольшей accuracy.

```
statistic = 1.32, p = 0.516
ранги: logreg 2.23, rf 1.83, hgb 1.93
```

p-value попарных сравнений (Неменьи):

	logreg	rf	hgb
logreg	1.000	0.517	0.690
rf	0.517	1.000	0.960
hgb	0.690	0.960	1.000

Поправка Холма на множественные сравнения

20 → 6
наивный

14 → 6
corrected

9 → 3
5×2cv

7 → 6
McNemar

Значимых сравнений из 45 до и после поправки. У наивного и corrected тестов остаются одни и те же 6: segment, spambase и все три пары на phoneme.

Код: поправка Холма

```
holm_rows = []
for t in TESTS:
    reject, p_adj, _, _ = multipletests(
        all_pairs[t], alpha=0.05, method="holm")
    holm_rows.append({
        "тест": t,
        "до": int((all_pairs[t] < 0.05).sum()),
        "после": int(reject.sum()),
    })
```

Поправка применяется к каждому тесту отдельно: 45 p-value на тест.

Результат:

тест	до	после
naive	20	6
corrected	14	6
5x2cv	9	3
McNemar	7	6

Проверка на данных с известным ответом

Два сценария

H_0: модели равны по построению. Доля «значимо» это ошибка первого рода, ожидаем около 5%.

H_1: вторая модель объективно хуже. Доля «значимо» это мощность.

500 повторов, 100, 200 и 400 объектов, логистическая регрессия на двух независимых «видах» признаков.

Пять процедур

1. Наивный t-test, 10-fold
2. Corrected t-test, 10-fold
3. 5×2cv
4. Наивный t-test, 30 повторных разбиений
5. Corrected t-test, 30 повторных разбиений

Код: симуляция

```
def p_from_differences(d, variance_factor, df):
    t = d.mean() / (d.std(ddof=1) * np.sqrt(variance_factor))
    return 2 * stats.t.sf(abs(t), df)

def make_two_view_data(rng, n_objects, shift1, shift2):
    y = rng.randint(0, 2, n_objects)
    sign = (2 * y - 1)[:, None]
    view1 = sign * shift1 + rng.randn(n_objects, 5)
    view2 = sign * shift2 + rng.randn(n_objects, 5)
    return np.hstack([view1, view2]), y

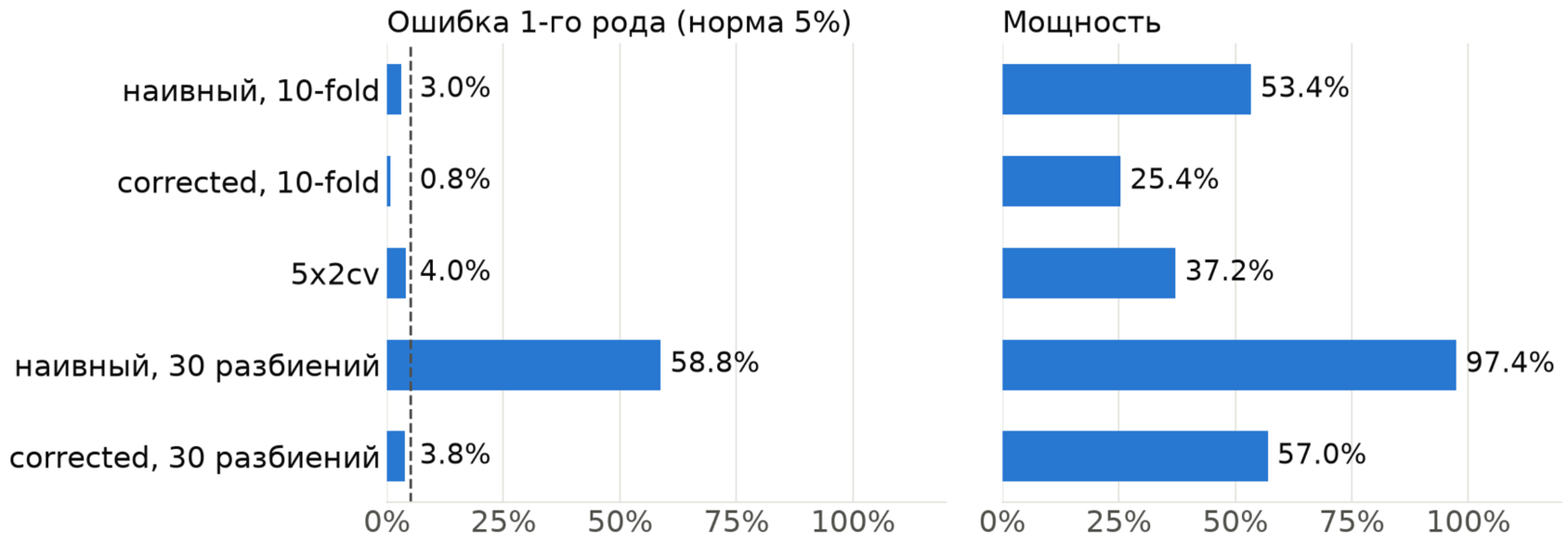
for scenario, shift2 in scenarios:
    p = Parallel(n_jobs=-1)(
        delayed(one_repetition)(r, 200, shift2)
        for r in range(500))
    rates[scenario] = (np.array(p) < 0.05).mean(axis=0)
```

variance_factor: 1/k для наивного теста, 1/k + 1/(k−1) для corrected.

Два «вида» признаков: модель A видит первый, модель B второй.

scenarios: H_0 (shift2 = 0.45, модели равны) и H_1 (shift2 = 0.30). one_repetition считает пять процедур, код сокращён.

Где наивный тест действительно ошибается



Повторные разбиения: 58.8% ложных находок при норме 5%. 10-fold: 3.0%. Поправка на 10-fold консервативна: мощность 25% против 53%. При 100 и 400 объектах картина та же.

Выводы

Выбор теста меняет вывод

Из 45 сравнений наивный тест находит 20 значимых, corrected 14, 5×2cv 9, McNemar 7.

Сильные различия видны всем

После Холма наивный и corrected оставляют одни и те же 6 сравнений (разрыв 2–16 п. п.).

Наивный тест ломается не везде

На повторных разбиениях 59% ложных находок, на 10-fold около 3%.

Лучшей модели в целом нет

Фридман на 15 датасетах: p = 0.52. Лидер меняется от датасета к датасету.

Ограничения

- 15 датасетов: мощность группового теста всё ещё ограничена.
- Гиперпараметры по умолчанию: цель работы это тесты, а не точность.
- На реальных данных истинная разница неизвестна, поэтому нужна симуляция.
- Симуляция: один дизайн (логрегрессия), 100–400 объектов.
- 5×2cv и McNemar приведены как медиана по нескольким разбиениям.

Источники

- Dietterich T. G. (1998). Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. Neural Computation, 10(7).
- Nadeau C., Bengio Y. (2003). Inference for the Generalization Error. Machine Learning, 52.
- Demšar J. (2006). Statistical Comparisons of Classifiers over Multiple Data Sets. JMLR, 7.
- García S., Herrera F. (2008). An Extension on «Statistical Comparisons of Classifiers over Multiple Data Sets» for all Pairwise Comparisons. JMLR, 9.
- Király F. J., Mateen B., Sonabend R. (2018). NIPS — Not Even Wrong? arXiv:1812.07519.

Спасибо за внимание

Вопросы?

Код, расчёты и графики: ноутбук first.ipynb